

Toegangsspraak

Klare taal voor toegangsbeleid – taal, editor en diagramprofiel in Omnium Studio

Mark Westbroek · augustus 2026 · info-paper, prototype-status

1 Waar dit over gaat

Toegangsbeleid leeft vandaag op twee plekken die elkaar niet kennen: in beleidsdocumenten (leesbaar, maar niet uitvoerbaar) en in autorisatieregels van systemen (uitvoerbaar, maar voor niemand leesbaar).

Toegangsspraak is een experiment om die kloof te sluiten: één beleidstekst in gewoon Nederlands die exact één betekenis heeft, verankerd is aan het onderliggende gegevensmodel, en verliesvrij heen en weer kan naar **ODRL** (JSON-LD, NLGov-profiel) – en via dat ODRL-spoor later naar runtime-engines als OPA/Rego, Cedar of XACML.

De hele taal hangt aan één kernzin:

<wie> mag <gegevens> <handeling> – of mag ... niet – [als <voorwaarden>] [waarbij: <verplichtingen>]

Een volledig (klein) beleid, met de kleuren van de zinsontleding uit de editor:

Beleid "Inzage inkomen bij schuldhulp".

Geldig vanaf 1 mei 2026.

Grondslag: de Wet gemeentelijke schuldhulpverlening.

Doel: "schuldhulpverlening".

Begrippen.

Een schuldhulpverlener is: iemand met rol "schuldhulpverlener".

Inkomensgegevens zijn: alle gegevens van het inkomen van een natuurlijk persoon.

Regel "inzage bij lopend dossier".

Een schuldhulpverlener mag de inkomensgegevens bekijken

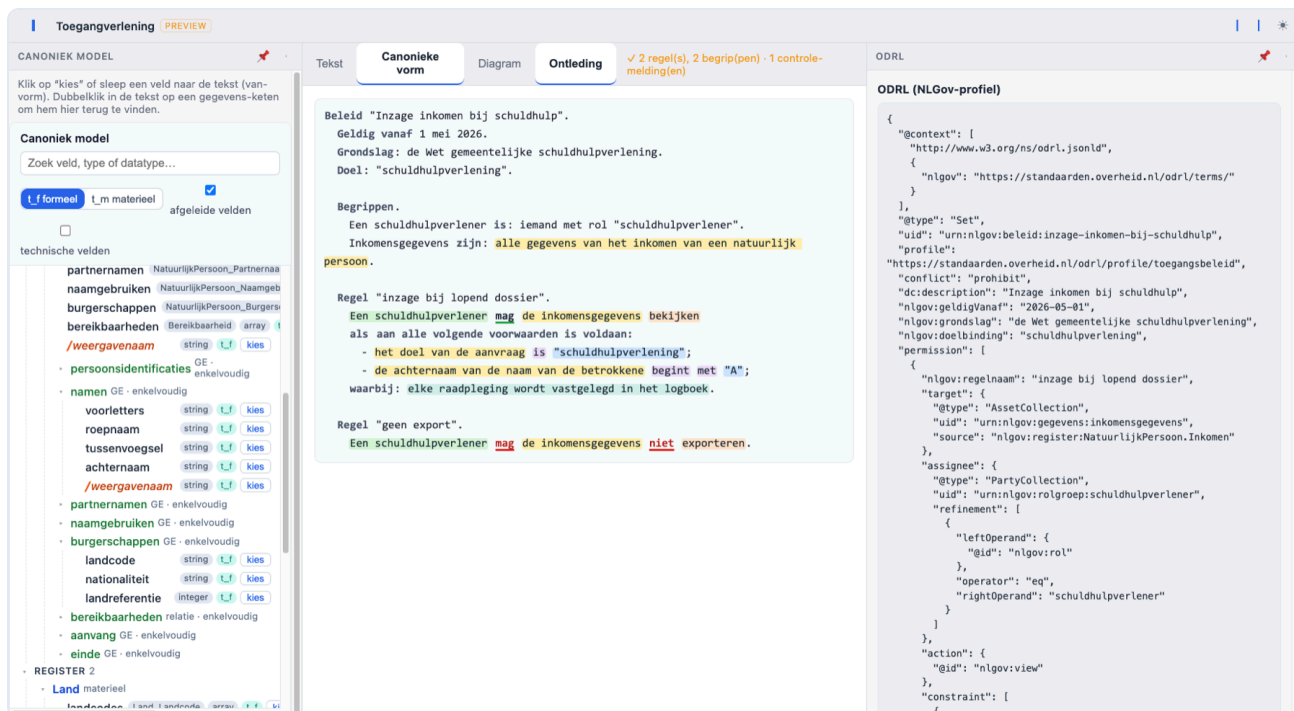
als aan alle volgende voorwaarden is voldaan:

- het doel van de aanvraag is "schuldhulpverlening";
- de achternaam van de naam van de betrokkene begint met "A";

waarbij: elke raadpleging wordt vastgelegd in het logboek.

Regel "geen export".

Een schuldhulpverlener mag de achternaam van de namen van een natuurlijk persoon niet exporteren.



Figuur 1. De Toegangsspraak-editor in Omnium Studio: links de modelboom van het canonieke model (klikken/slepen voegt de van-vorm in), in het midden de beleidstekst met zinsontleding en statusregel, rechts de live meegenereerde ODRL-vorm.

2 Hoe parse je de zinnen precies? Zit daar een formele grammatica onder?

Ja – er ligt een formele grammatica onder, en er komt geen taalmodel of NLP-gokwerk aan te pas.

Toegangsspraak is een *gecontroleerde natuurlijke taal* (CNL), in dezelfde familie als RegelSprak van de Belastingdienst: het leest als Nederlands, maar elke zin volgt een vast patroon uit een vaste grammatica en heeft precies één betekenis. Het parsen is daardoor volledig deterministisch – dezelfde tekst levert altijd dezelfde boom, en wat niet in de grammatica past is een foutmelding (in klare taal, met regel- en kolomnummer), geen interpretatie.

2.1 De grammatica (EBNF)

De kern in compacte EBNF (leesvorm; de editor schrijft altijd de canonieke vorm terug):

```

beleid      = kop , [ begrippen ] , { regel } ;
kop          = "Beleid" , naam , "." , { kopregel } ;
kopregel     = "Geldig vanaf" , datum , [ "tot" , datum ] , "."
              | "Grondslag:" , tekst , "." | "Doel:" , string , "." ;

begrippen    = "Begrippen." , { begripsdef } ;
begripsdef   = [ lidwoord ] , naam , "is:" , "iemand" , kenmerken , "."      (* wie
*)
              | [ lidwoord ] , naam , ("is:"|"zijn:") , wat , [ "waarvan" , voorwaarde ]
, "." ; (* wat *)
kenmerken    = "met" , kenmerk , string , { "en" , kenmerk , string } ;

regel        = "Regel" , naam , "." , kernzin ;
kernzin      = wie , "mag" , wat , [ "niet" ] , actie ,
              [ "als" , voorwaardeblok ] ,
              [ "waarbij:" , plicht , { ";" , plicht } ] , "." ;

wie          = lidwoord , begripsnaam | "iemand" , kenmerken ;
wat          = [ lidwoord ] , begripsnaam
              | "alle gegevens van" , verwijzing
              | verwijzing ;

voorwaardeblok = voorwaarde
              | kwantor , ":" , { "-" , ( voorwaarde | voorwaardeblok ) , [ ";" ] } ;
kwantor      = "aan alle volgende voorwaarden is voldaan"                  (* AND
*)
              | "aan ten minste één van de volgende voorwaarden is voldaan" (* OR
*)
              | "aan precies één van de volgende voorwaarden is voldaan" ; (* XOR
*)
voorwaarde    = term , vergelijking , [ rechterdeel ]                      (*
stelling én bijzin, §2.4 *)
              | "er" , [ "is" ] , [ "geen" ] , omschrijving , [ "voor" , verwijzing ] , [
"is" ] ;
term          = string | getal | datum | verwijzing ;

verwijzing    = groep , { "van" , groep }      (* van-keten; laatste groep = basis *)
              | pad ;                          (* technische shorthand:
NatuurlijkPersoon.Naam.achternaam *)
groep         = lidwoord , woord , { woord } ;
basis         = "de aanvrager" | "de betrokkene" | "de aanvraag" | "de gegevens" (*
ankers *)
              | typenaam ;

vergelijking  = (* uit het operator-register, §2.3 *) ;
actie         = (* uit het actie-register: bekijken, opvoeren, veranderen, exporteren, ...
*) ;
plicht        = (* uit het plichten-register: "elke raadpleging wordt vastgelegd in het
logboek", ... *) ;

```

De grammatica is **LL(1)-achtig**: elk alternatief is op het eerste woord (of token) te onderscheiden, zodat de parser nooit hoeft te backtracken over structuur. Interpunctie is betekenisdragend, net als in RegelSprak: de opsomming met streepjes is de boolese structuur (AND/OR/XOR, nestbaar via insprong), waardoor de klassieke en/of-ambigüiteit van lopende tekst niet kán ontstaan.

2.2 De pijplijn: tokenizer → recursive descent → AST

De implementatie is een **handgeschreven tokenizer + recursive-descent parser** (plain JavaScript, geen parser-generator, geen dependencies). De tokenizer kent vijf tokensoorten – *woord*, *getal*, *string* (tussen aanhalingstekens, ook typografische), *pad* (het technische registerpad zoals `NatuurlijkPersoon.Naam.achternaam`) en *leesteken* – plus het opsommingsstreepje, dat zijn insprong meedraagt: daarmee bepaalt de parser de nesting van voorwaardeblokken. Elk token onthoudt zijn bronpositie.

De parser volgt het klassieke recept *één functie per grammaticaregel*: `parseBeleid → parseBegrip / parseRegel → parseWat / parseVoorwaardeblok → parseVoorwaarde → parseTerm → parseVerwijzing`. Het resultaat is een **AST** (abstracte syntaxboom) die positie-onafhankelijk is, plus een losse lijst *spans*: bronposities per zinsdeel-soort (subject, gegevens, handeling, vergelijking, waarde, plicht, modaliteit). Die spans voeden de zinsontleding-weergave in de editor – de kleuren in het voorbeeld hierboven komen rechtstreeks uit de parser, niet uit een aparte highlighter. Doordat de spans buiten de AST blijven, is de AST zelf exact vergelijkbaar over round-trips heen.

2.3 Open slots: operatoren, handelingen en plichten zijn data, geen grammatica

De grammatica kent alleen de *slots* "vergelijking", "handeling" en "plicht"; de invulling komt uit **registers** (gewone datalijsten). De kernset bevat o.a. `is`, `is niet`, `is groter/kleiner dan`, `is ten minste/hogste`, `begint met`, `eindigt op`, `bevat`, `is een van (...)`, `ligt tussen ... en ...`, `is bekend/onbekend` – elk met zijn ODRL-tegenhanger (`odrl:eq`, `nlgov:beginMet`, ...) en de waardetypen waarop hij past. De parser matcht operatoren *longest match first*, zodat "is kleiner dan" wint van "is".

Een domeinprofiel registreert extra termen zonder één grammaticaregel te wijzigen – er is een geo-profiel meegeleverd ("valt geheel binnen", "overlapt", → `geo:within`, `geo:intersects`). Dat is bewust het **ODRL-Profile-mechanisme, doorgetrokken naar de taal**.

2.4 Nederlandse woordvolgorde: stelling én bijzin

Correct Nederlands vraagt na "als" de bijzinsvolgorde ("... als de taal niet "nl" is"), terwijl een opsommingsregel de stellingsvorm heeft ("de taal *is niet* "nl""). De parser accepteert beide: herkent hij na de linksterm geen operator, dan haalt hij het werkwoord aan het einde van de voorwaarde naar voren en parseert het geheel opnieuw als stellingsvorm. Herformatteren normaliseert naar de canonieke vorm per context (bijzin na als/waarvan, stelling in opsommingen). Eén betekenis, twee leesbare schrijfwijzen.

2.5 Semantiek: verankering aan het gegevensmodel

Na het parsen volgt de semantische stap, en die maakt de taal meer dan een grammatica-oefening. Naar gegevens verwijst je in de **van-vorm**: "de achternaam van de naam van een natuurlijk persoon" – het woord "van" volgt de compositie in het model in omgekeerde richting. Zo'n keten wordt **geresolvd tegen het echte metamodel** (de schema-API van het register): de keten mag verkort worden zolang hij eenduidig blijft ("de achternaam van een natuurlijk persoon" volstaat als er maar één veld `achternaam` onder `NatuurlijkPersoon` hangt). Is de verwijzing dubbelzinnig – `achternaam` bestaat zowel onder `Naam` als onder `PartnerNaam` – dan komt er een controle-melding en biedt de editor de kandidaten aan; één klik vervangt de keten door de eenduidige vorm (figuur 2).

Dezelfde stap doet **typebewaking**: "begint met" kan alleen met tekst, een datumveld vergelijk je niet met een getal, en enum-velden alleen met hun toegestane waarden – met foutmeldingen die zelf ook klare taal zijn ("'*begint met*' kan alleen met tekst; '*geboortedatum*' is een datum. Bedoelde je '*is eerder dan*'?"). De ODRL-uitvoer gebruikt de geresolvide registerpaden.

NatuurlijkPersoon

materieel

persoonsidentificaties

NatuurlijkPersoon_Pe

namen

NatuurlijkPersoon_Naam

array

t.f

partnernamen

NatuurlijkPersoon_Partnernaam

naamgebruiken

NatuurlijkPersoon_Naamgebruik

burgerschappen

NatuurlijkPersoon_Burgersch

bereikbaarheden

Bereikbaarheid

array

t.f

/weergavenaam

string

t.f

kies

persoonsidentificaties

GE

enkelvoudig

namen

GE

enkelvoudig

voorletters

string

t.f

kies

roepnaam

string

t.f

kies

tussenvoegsel

string

t.f

kies

achternaam

string

t.f

kies

NatuurlijkPersoon.namen.tussenvoegsel

/weergavenaam

string

t.f

kies

partnernamen

GE

enkelvoudig

achternaam

string

t.f

kies

naamgebruiken

GE

enkelvoudig

burgerschappen

GE

enkelvoudig

landcode

string

t.f

kies

nationaliteit

string

t.f

kies

landreferentie

integer

t.f

kies

bereikbaarheden

relatie

enkelvoudig

regel "inzage bij lopend dossier".

Een schuldhulpverlener mag de inkomensgegevens bekijken

als aan alle volgende voorwaarden is voldaan:

- het doel van de aanvraag is "schuldhulpverlening";

- de achternaam van de naam van de betrokkene begint met "A";

waarbij: elke raadpleging wordt vastgelegd in het logboek.

Regel "geen export".

Een schuldhulpverlener mag de achternaam van een natuurlijk persoon niet

exporteren.

Gegevens: registerpad NatuurlijkPersoon.achternaam — niet gevonden in het metamodel.

Tab $\text{Ctrl}+\text{Space}$ de achternaam van de namen van een natuurlijk persoon

de achternaam van de partnernamen van een natuurlijk persoon

Controle: Begrip "Inkomensgegevens": Onder natuurlijk persoon is geen gegevensgroep "inkomen" bekend.

Controle: Regel "geen export": "de achternaam van een natuurlijk persoon" is dubbelzinnig: het kan "NatuurlijkPersoon.namen.achternaam", "NatuurlijkPersoon.partnernamen.achternaam" zijn. Gebruik de volledige keten.

Figuur 2. Keten-resolutie tegen het metamodel: een dubbelzinnige verwijzing levert een controle-melding; klikken op de juiste kandidaat (onderin of in de modelboom) vervangt de keten in de policytekst.

2.6 Round-trip als wet

De tekst is de bron; de canonieke vorm en de ODRL-weergave zijn projecties van dezelfde AST. De geteste garantie is: $\text{render}(\text{parse}(t))$ is de canonieke schrijfwijze van t , en $\text{parse}(\text{render}(b)) = b$ voor elk beleid. Diezelfde eis geldt langs de diagramkant (§4). Daarmee is álles wat in het register staat per constructie leesbaar – er kan geen policy bestaan die niet als Nederlandse tekst te tonen is.

3 Van tekst naar ODRL (NLGov-profiel)

De afbeelding op ODRL is deterministisch en behoudt de structuur één-op-één:

Toegangsspraak	ODRL (JSON-LD, NLGov-profiel)
Beleid + kopregels (geldig, grondslag, doel)	Set met <code>nlgov:geldigVanaf</code> , <code>nlgov:grondslag</code> , <code>nlgov:doelbinding</code>
mag / mag ... niet	<code>permission</code> / <code>prohibition</code>
begrip "wie" (iemand met rol ...)	<code>PartyCollection</code> met <code>refinement</code>
begrip "wat" / van-keten	<code>Asset</code> / <code>AssetCollection</code> met registerpad als <code>source</code>
handeling (bekijken, exporteren, ...)	<code>action</code> (<code>nlgov:view</code> , <code>nlgov:export</code> , ...)
voorwaardeblok (alle / ten minste één / precies één)	<code>constraint</code> / <code>LogicalConstraint</code> (and / or / xone)
waarbij: plichten	<code>duty</code> (<code>nlgov:log</code> , ...)
vaste conflictregel: verbod wint, default deny	<code>conflict: prohibit</code>

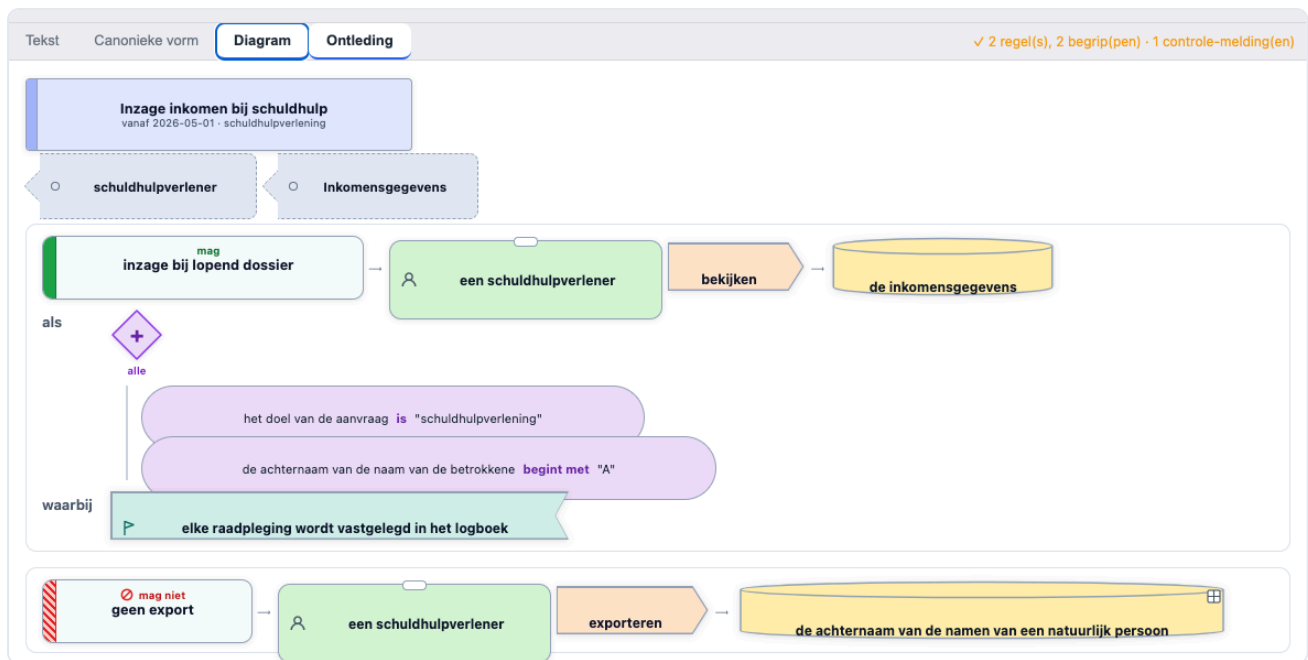
Zo wordt de regel "inzage bij lopend dossier" uit §1 dit fragment (ingekort):

```
"permission": [{
  "nlgov:regelnaam": "inzage bij lopend dossier",
  "target": { "@type": "AssetCollection", "uid": "urn:nlgov:gegevens:inkomensgegevens",
    "source": "nlgov:register:NatuurlijkPersoon.Inkomen" },
  "assignee": { "@type": "PartyCollection", "uid":
    "urn:nlgov:rolgroep:schuldhelpverlener",
    "refinement": [{ "leftOperand": {"@id": "nlgov:rol"}, "operator": "eq",
      "rightOperand": "schuldhelpverlener" } ] },
  "action": { "@id": "nlgov:view" },
  "constraint": [
    { "leftOperand": {"@id": "nlgov:doelbinding"}, "operator": "eq",
      "rightOperand": "schuldhelpverlening" },
    { "leftOperand": {"@id": "nlgov:betrokkene:naam.achternaam"},
      "operator": "nlgov:begintMet", "rightOperand": "A" } ],
  "duty": [{ "action": {"@id": "nlgov:log"} } ]
}]
```

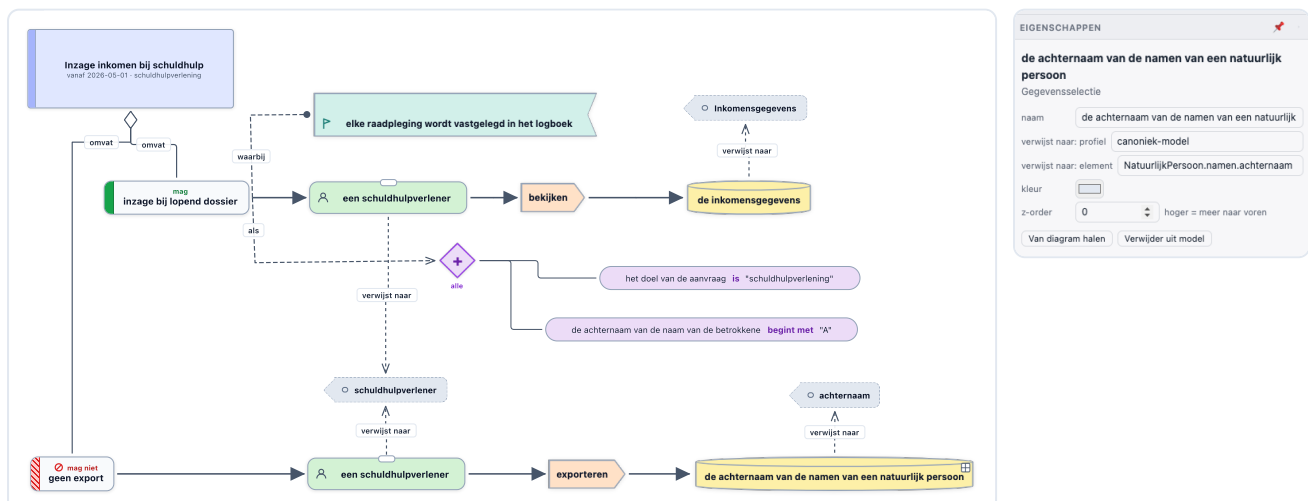
4 Hetzelfde beleid als diagram: het toegangsregel-profiel

Omnium Studio heeft een generieke diagrammotor met *profielen* (vormtaal + regels per notatie). Voor toegangsbeleid bestaan er twee weergaven op dezelfde AST:

- **Regelkaarten** – een read-only tab naast de tekst: per regel het subject, de handeling, de gegevens, de voorwaardepoort (alle / ten minste één / precies één) en de plichten, in dezelfde kleuren als de zinsontleding.
- **Het bewerkbare model** – de regels als echt diagram-model in de modelleeromgeving. Daar kun je het beleid grafisch aanpassen en er de tekst en ODRL weer uit genereren (*Publiceer naar Modelleren / Lees terug uit Modelleren*; de layout blijft bij publiceren behouden). Elke gegevensselectie in het diagram draagt in zijn eigenschappen de verwijzing naar het element in het canonieke model – bv. `NatuurlijkPersoon.namen.achternaam` in het profiel `canoniek-model`.

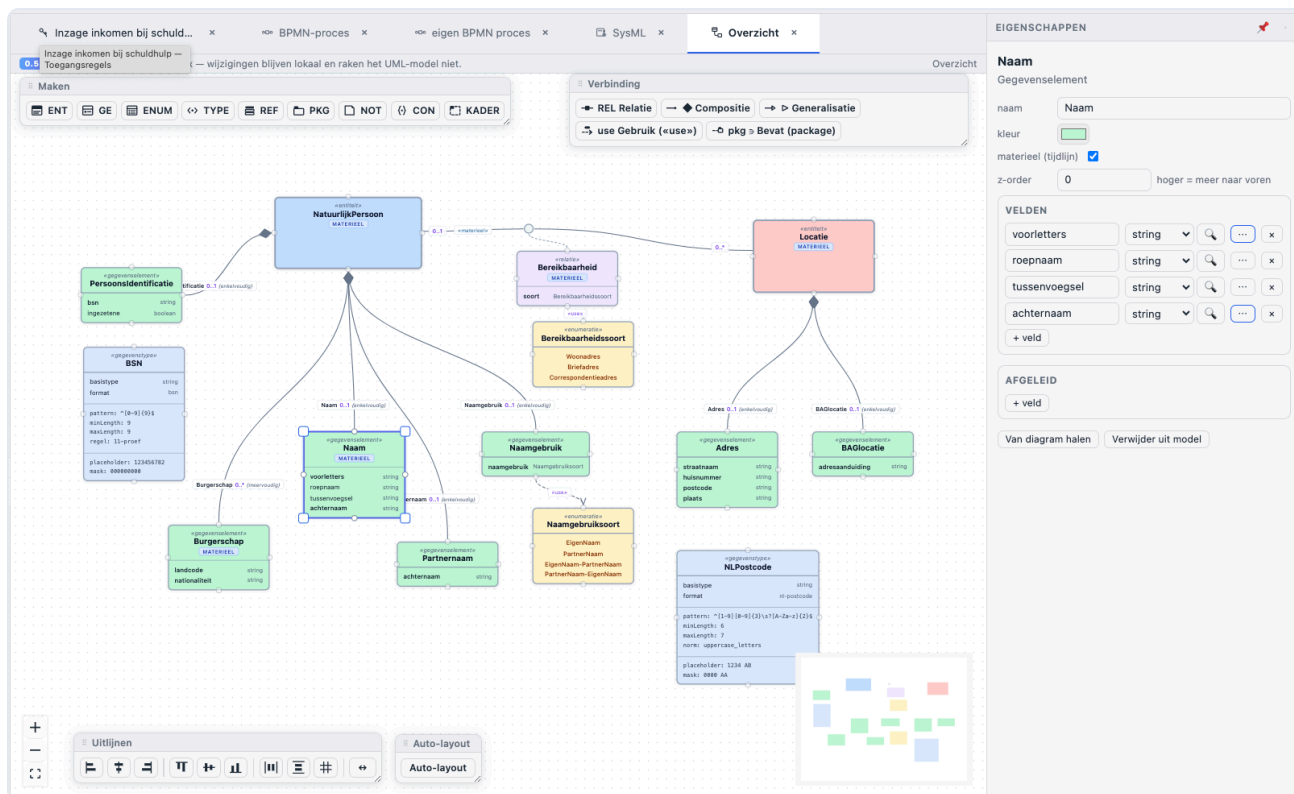


Figuur 3. Regelkaarten: toestemming (groen "mag") en verbod (rood "mag niet"), met voorwaardepoort en plicht.



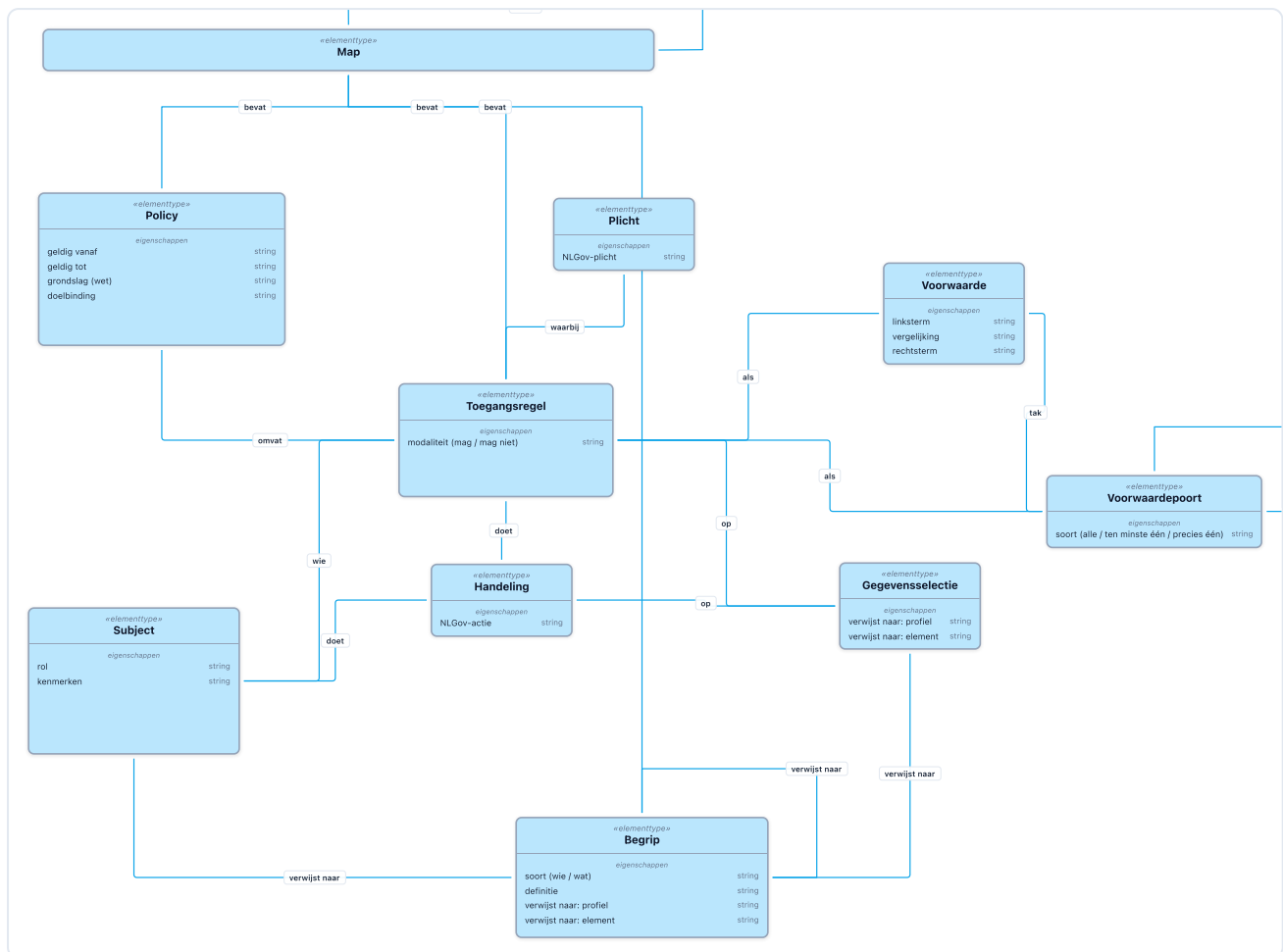
Figuur 4. Het bewerkbare model: beide regels met subject, handeling, gegevensselectie, voorwaardepoort en plicht. Rechts de eigenschappen van de gegevensselectie "de achternaam van de namen van een natuurlijk persoon": de link naar profiel `canoniek-model`, element `NatuurlijkPersoon.namen.achternaam`.

Het canonieke gegevensmodel zelf staat in dezelfde tool, in een gespecialiseerde UML-vorm (entiteiten, gegevenselementen, gegevenstypen met patronen en enumeraties). Dat model is tegelijk de bron waaruit elders een bitemporeel register wordt gegenereerd – maar voor de policytaal is het vooral: de plek waar de van-ketens naartoe resolen.



Figuur 5. Het canonieke model in gespecialiseerde UML – het resolutiedoel van de van-vorm én de bron voor registratie.

Ook het profiel zelf is model: een metamodel van elementtypen (Policy, Toegangsregel, Subject, Handeling, Gegevensselectie, Voorwaarde, Voorwaardepoort, Plicht, Begrip) met hun verbindingsregels (omvat, wie, doet, op, als, waarbij, verwijst naar). Profielen worden in de Studio getekend als diagram en dan geactiveerd tot werkende notatie – het toegangsregel-profiel is dus met zijn eigen mechanisme gebouwd.



Figuur 6. Het metamodel onder het profiel: Policy omvat Toegangsregels (wie / doet / op / als / waarbij), voorwaarden nestbaar via de Voorwaardespoort, en Begrip en Gegevensselectie verwijzen naar het canonieke model. "Map" is alleen een structurelement.

5 De editor in het kort

- **Live parsen** met foutmeldingen in klare taal (regel + kolom) en een statusregel ("✓ 2 regel(s), 2 begrip(pen) · 1 controle-melding(en)").
- **Vier weergaven op één bron:** Tekst, Canonieke vorm (geherformatteerd), Diagram (regelkaarten) en Ontleding (zinsdelen gekleurd vanuit de parser-spans).
- **Autocomplete, twee kanten op:** een woord typen stelt van-vormen voor; "de naam van " typen stelt de bases voor die zo'n veld hebben. Kort (verkorte keten) of volledig invoegen; binnen een bestaande keten vervangt een suggestie de hele keten.
- **Modelboom-koppeling:** velden uit de modelboom klikken of slepen voegt de van-vorm in; dubbelklik op een keten in de tekst toont het registerpad en focust het element in de boom.
- **Controle-meldingen** (niet-blokkerend): keten-resolutie, dubbelzinnigheid, typebewaking, enum-waarden.
- **Menu Beleid:** voorbeeld laden, herformatteren, ODRL-export, publiceren naar / teruglezen uit de modelleromgeving, koppelen aan ArchiMate (begrippen → Business object/rol, grondslag → Constraint, doel → Goal).

6 Status en vervolg

Toegangsspraak is een **werkend prototype** (onderdeel van Omnium Studio, status concept): taal, parser, renderer, ODRL-export, metamodel-controle, editor en diagramprofiel werken en zijn met unit-tests afgedekt (grammatica, round-trip, ODRL, resolutie, suggesties). Het is nadrukkelijk *geen* runtime-engine – de PDP beslist – en nog geen productiestandaard. Op de rol staan: bitemporele registratie van beleidsteksten (zodat "welk beleid gold er op 1 maart?" een registrervraag wordt), de vertalers ODRL → Rego/Cedar, en een nette subgrammatica voor plichten.

Reacties zijn welkom – juist ook op de taalkeuzes: welke zinnen wil een beleidsmaker kunnen schrijven die nu nog niet passen?